

GarudaChip: A Locally-Hosted Recursive-Language-Model Agent for Prompt-to-GDSII ASIC Design with Hybrid Retrieval-Augmented Generation

Ade Irman
Pindad Military Ltd.
Bandung, Indonesia
adeirman2705@gmail.com

Abstract—GarudaChip is a fully local, agentic electronic-design-automation (EDA) system that turns a natural-language hardware request into synthesizable RTL and, ultimately, a GDSII layout on an open-source process design kit, without any cloud large-language-model (LLM) service. The flow is a fixed pipeline graph—plan, research, generate, decompose, test, simulate, lint, harden—in which every reasoning node is realized as a recursive-language-model (RLM) deep agent equipped with planning, file, web-research and recursive sub-call tools. To keep a small 9-billion-parameter local model effective, GarudaChip pairs a durable Postgres/pgvector plus object-storage knowledge base with a hybrid retrieval pipeline that fuses dense and lexical search by reciprocal rank fusion and reranks with a local cross-encoder, and it exposes a hardware-aware context-window control sized to the user’s GPU memory. An autoresearch loop lets the agent learn from each run. Rather than copying reference designs verbatim, the generator studies retrieved knowledge and authors its own parameterized modules. We demonstrate an RV32I soft core taken to a clean multi-corner GDSII sign-off on the GF180MCU 180 nm node and a coarse-grained reconfigurable array taken through simulation and synthesis.

Index Terms—agentic EDA, RTL generation, large language models, retrieval-augmented generation, reranking, recursive language models, GDSII, GF180MCU, local LLM.

I. INTRODUCTION

The cost and scarcity of digital-design expertise motivate tools that lower the barrier from intent to silicon. Recent LLMs can draft Verilog, but production use in defense and sovereign-technology settings raises two hard constraints: design data must never leave the premises, and the available compute is modest. Cloud LLM APIs violate the first; large local models violate the second. GarudaChip targets exactly this regime: a single workstation, an open-source toolchain, and a small local model.

Our central claim is that a small local model becomes a capable hardware designer when it is embedded in the right scaffolding: a deterministic flow graph that guarantees the right step runs at the right time; deep agents that plan and use tools instead of emitting code in one shot; a knowledge base that remembers every prior design, reference and verified fix; and a retrieval pipeline good enough that the few thousand tokens

the model can attend to are the most relevant ones available. Fig. 1 gives the overview; we then detail the deep-agent node (Fig. 2), the autoresearch loop (Fig. 3), the deep-agent roster (Fig. 4), and hybrid RAG (Fig. 5), before reporting case-study results.

II. RELATED WORK

LLM-assisted RTL design has been explored by domain-adapted models such as ChipNeMo [1] and by benchmarks like VerilogEval [2] and RTLLM [3], which mostly evaluate single-shot code generation. Agentic methods add planning and tool use; Reflexion-style self-improvement loops [4] let an agent learn from execution feedback. Recursive language models [5] decompose a task so a model interacts with an external context it never fully loads. On the retrieval side, RAG [6] augments generation with fetched context; hybrid search and reciprocal rank fusion (RRF) [7] combine dense and lexical rankings, and cross-encoder rerankers [8] sharply improve precision. GarudaChip integrates these threads into an end-to-end, local prompt-to-GDSII flow with an open-source hardening back end (LibreLane/OpenROAD [11], [12] on GF180MCU [13]).

III. SYSTEM ARCHITECTURE

GarudaChip is a fixed directed graph of pipeline stages (Fig. 1). The graph is deterministic: it always plans before it generates and always simulates and lints before it hardens, with an error-to-fix arc that returns failing designs to generation. Determinism at the graph level supplies the guardrails a small model needs; flexibility lives inside each node.

A. Deep agents per node

Each reasoning stage—planner, researcher, generator, testbench writer, and the module and testbench correctors—executes as a deep agent (Fig. 2): it writes a to-do list, reads and greps files in slices rather than swallowing them, issues focused recursive sub-calls (`llm_query`) to a fresh model instance, and can search the web. Mechanical stages (decompose, write, simulate, lint, harden) are deliberately kept free of the LLM and run regex/Icarus/Verilator/LibreLane so

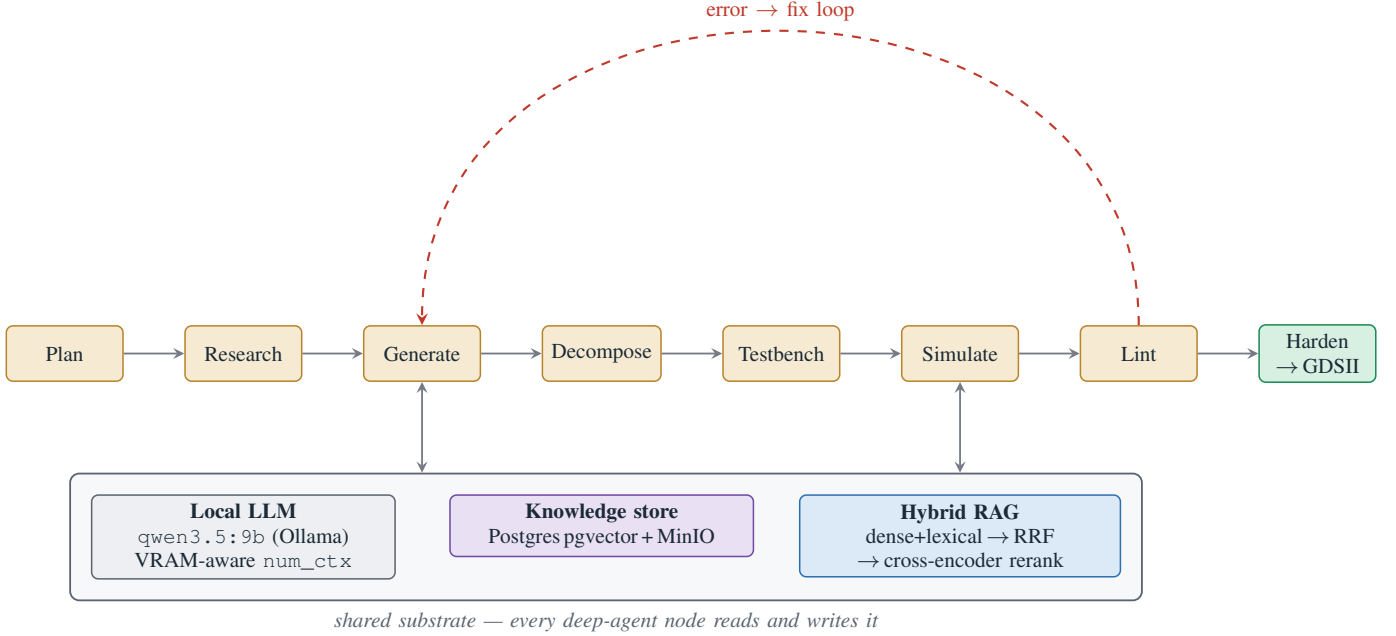


Fig. 1. GarudaChip architecture. A fixed, deterministic pipeline graph (top) drives the prompt-to-GDSII flow with an error-to-fix feedback loop; each reasoning node executes as an RLM deep agent (Fig. 2) over a shared substrate (bottom): a local model with a VRAM-aware context window, a durable knowledge store, and a hybrid retrieval stack.

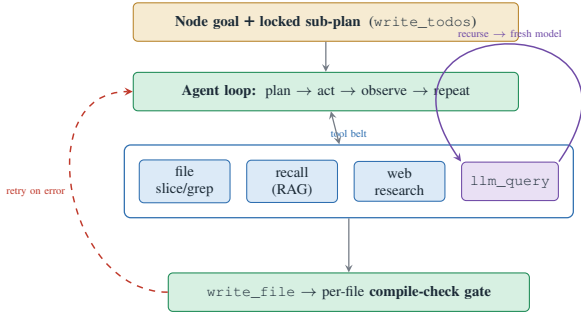


Fig. 2. Anatomy of one RLM deep agent. A node plans a to-do list, then loops over a shared tool belt—file slicing, RAG recall, web research, and `llm_query`, the recursive leaf that delegates a focused sub-task to a fresh model instance. Every file write passes a compile-check gate before the node may finish.

generated code is preserved byte-for-byte. Fig. 4 lists the agent roster.

B. Durable knowledge store

Long-term memory is a Postgres database with the pgvector extension for embeddings and a MinIO object store for raw artifacts—RTL, references, papers, waveforms and GDS. Every finished design is ingested, so the system improves with use; a committed seed ships with the repository so a fresh clone is never empty. The store is defensive: if it is unavailable the agent falls back to a pure-local file flow.

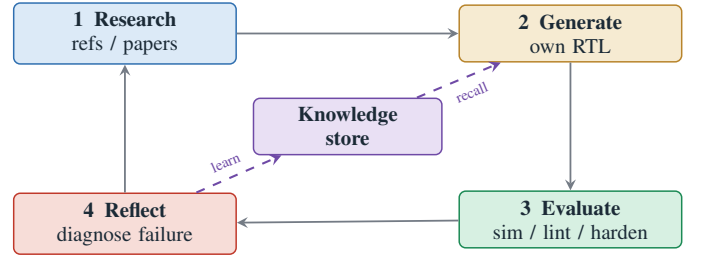


Fig. 3. Autoresearch self-improvement loop. The agent researches references, authors RTL, evaluates it with the open-source tools, and reflects on failures; verified error-to-fix lessons and generation methods are written to the knowledge store and recalled on the next run, so quality compounds over time.

C. Local model and context window

All generation uses a local Ollama-served `qwen3.5:9b` model. Because the key-value cache of such a model costs on the order of 56 KB per token, the usable context window is bounded by GPU memory, not by the model. GarudaChip detects the available VRAM (or system RAM under CPU inference) and exposes a context-window slider whose maximum is the key-value-cache budget after reserving the model weights, so a user can trade context for memory and never select a window that will not fit.

IV. GENERATE-FROM-KNOWLEDGE, AUTORESEARCH, AND HYBRID RAG

A naive agent that clones the nearest reference repository produces designs that are merely renamed copies, sometimes

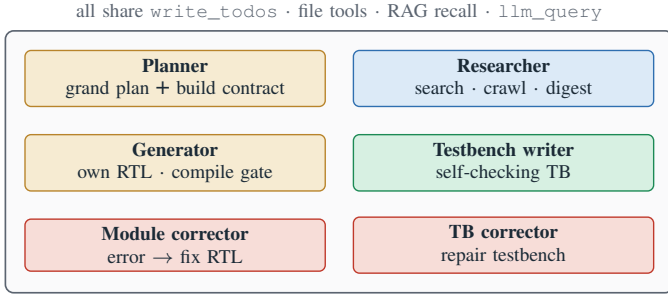


Fig. 4. The deep-agent roster. Each pipeline node is a specialized agent with its own goal, drawing on a common tool belt; LLM-free stages (decompose, write, simulate, lint, harden) are omitted.

hundreds of near-identical blocks. GarudaChip instead treats retrieved repositories and papers as knowledge to study, not code to paste. The architect derives its own module map and is instructed to collapse repetition into one parameterized module instantiated N times; the generator then authors each module itself, grounded in retrieved knowledge and guarded by per-file compile checks and a completeness gate.

A. Autoresearch

GarudaChip closes a self-improvement loop (Fig. 3). After each run, verified error-to-fix pairs and effective generation methods are written to the knowledge store; on subsequent runs they are recalled and injected before generation, so the agent stops repeating past mistakes. This Reflexion-style loop turns the durable store into accumulating expertise rather than a static index.

B. Deep-agent roster

Fig. 4 summarizes the specialized agents. They share the same tool belt and substrate but differ in goal and in which tools dominate: the researcher leans on web search and crawling, the generator on knowledge recall and the compile-check gate, and the correctors on error-grounded retrieval.

C. Hybrid retrieval

Under a tight context budget, retrieval quality is decisive. Recall is hybrid (Fig. 5): a dense pgvector cosine search and a lexical Postgres full-text search are each run broadly, merged by reciprocal rank fusion, and then reranked by a local cross-encoder that scores every query–passage pair jointly. Only the few highest-scoring chunks enter the prompt, maximizing signal per token. Each stage degrades gracefully: without the full-text index the system is dense-only, and without the reranker it falls back to fusion order.

As a qualitative check, for the query “8-bit ALU with carry flag” the cross-encoder scored a matching ALU passage far above an unrelated UART passage, and hybrid recall returned ALU register-transfer code and design notes ahead of weaker matches, confirming that the reranker reorders fused candidates by true relevance.

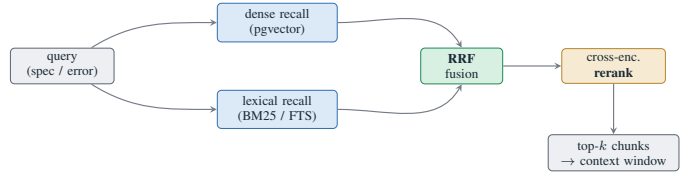


Fig. 5. Hybrid retrieval: dense and lexical recall are fused by RRF, reranked by a local cross-encoder, and only the top- k chunks are sent to the model.

TABLE I
END-TO-END CASE-STUDY RESULTS ON THE OPEN-SOURCE GF180MCU FLOW.

Metric	FazyRV RV32I	CGRA 2×2
RTL files / lines	23 / 4332	multi / param.
RTL simulation	PASS	PASS
Synthesis (Yosys)	clean	0 err / 0 warn
GDSII (GF180MCU)	produced	RTL + synth.
Die area	5304 μm^2	—
Placed instances	207	—
Lint errors	0	0
Setup/hold violations	0 (tt/ss/ff)	—

V. RESULTS

We report two case studies produced end-to-end by the system and summarized in Table I. The knowledge store currently holds 1,125 indexed items (700 code, 206 reusable IP, 63 GDS, 49 verified fixes, 23 papers and others), and the bundled IP library exposes 92 blocks spanning CPU cores, memories, buses, peripherals, DSP and cryptography.

A. RV32I soft core

From the prompt “create a FazyRV 8-bit RISC-V”, GarudaChip produced a 23-file, 4,332-line RTL core that passes self-checking simulation and hardens to GDSII on GF180MCU. Static timing analysis reported zero setup and zero hold violations across the typical, slow and fast corners (nom_tt 25 °C/5.0 V, nom_ss 125 °C/4.5 V, nom_ff −40 °C/5.5 V); the layout has a die area of 5304 μm^2 with 207 placed instances and no lint errors—a clean, sign-off-ready macro.

B. Coarse-grained reconfigurable array

A 2×2 CGRA accelerator exercises the parameterized generate-from-knowledge path: a processing-element tile is described once and instantiated across the array rather than emitted as repeated copies. The design passes simulation and synthesizes cleanly in Yosys with zero errors and zero warnings, providing the netlist for subsequent placement and routing.

VI. DISCUSSION AND LIMITATIONS

GarudaChip shows that scaffolding, not model scale, is the lever for local agentic EDA: a 9B model with hybrid reranked retrieval and a deterministic flow reaches GDSII sign-off on a real PDK. Limitations remain. Throughput is bounded by

local inference, so complex designs require several generate–verify iterations; the current evaluation is case-study rather than a large benchmark sweep; and hardening is validated on GF180MCU only. The reranker adds a one-time model download and a small per-query cost. Future work includes a quantitative VerilogEval/RTL2LLM comparison, contextual-retrieval chunk augmentation at ingestion, and power- and area-directed optimization loops.

VII. CONCLUSION

We presented GarudaChip, a fully local recursive-language-model agent that converts natural-language intent into RTL and GDSII on an open-source toolchain. By combining a deterministic pipeline of deep agents, an autoresearch self-improvement loop, a durable knowledge store, hybrid reranked retrieval and a VRAM-aware context window, the system makes a small on-premises model a practical hardware designer, achieving a clean multi-corner sign-off for an RV32I core and simulation-plus-synthesis closure for a reconfigurable accelerator.

ACKNOWLEDGMENT

The author thanks Pindad Military Ltd. for supporting sovereign, on-premises design-automation research. GarudaChip builds on open-source projects including Ollama, pgvector, MinIO, Yosys, OpenROAD, LibreLane and the GF180MCU PDK.

REFERENCES

- [1] M. Liu *et al.*, “ChipNeMo: Domain-adapted LLMs for chip design,” arXiv:2311.00176, 2023.
- [2] M. Liu *et al.*, “VerilogEval: Evaluating large language models for Verilog code generation,” in *Proc. ICCAD*, 2023.
- [3] Y. Lu *et al.*, “RTL2LLM: An open-source benchmark for design RTL generation with large language models,” in *Proc. ASP-DAC*, 2024.
- [4] N. Shinn *et al.*, “Reflexion: Language agents with verbal reinforcement learning,” in *Proc. NeurIPS*, 2023.
- [5] MIT CSAIL, “Recursive Language Models,” arXiv preprint, 2025.
- [6] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, 2020.
- [7] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. SIGIR*, 2009.
- [8] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP*, 2019.
- [9] A. Yang *et al.*, “Qwen technical report,” arXiv:2412.15115, 2024.
- [10] C. Wolf, “Yosys open synthesis suite,” <https://yosyshq.net>, 2023.
- [11] T. Ajayi *et al.*, “OpenROAD: Toward a self-driving, open-source digital layout implementation tool chain,” in *Proc. GOMACTech*, 2019.
- [12] Efabless and contributors, “LibreLane / OpenLane RTL-to-GDSII flow,” 2024.
- [13] GlobalFoundries and Google, “GF180MCU open-source PDK,” 2022.